

Ini adalah penjelasan mendalam mengenai kode C# yang Anda berikan. Kode ini berfungsi sebagai **Pabrik (Factory)** atau generator untuk membuat *kernel* konvolusi.

Dalam pengolahan citra digital, **Kernel** (atau matriks filter) adalah matriks kecil angka yang "digeser" di atas gambar untuk memanipulasi pixel, seperti membuat gambar menjadi buram (*blur*), tajam (*sharpen*), atau mendeteksi tepi (*edge detection*).

Berikut adalah penjelasan detail per bagian dan logika baris kodenya:

1. Struktur Data (KernelData)

Bagian ini mendefinisikan "wadah" untuk menyimpan data filter yang akan digunakan.

C#

```
public class KernelData
{
    public double[,] Matrix { get; set; }
    public double Divisor { get; set; }
    public int Offset { get; set; }

    public KernelData(double[,] m, double d, int o)
    {
        Matrix = m;
        Divisor = d;
        Offset = o;
    }
}
```

- **public double[,] Matrix:** Menyimpan grid angka (matriks 2 dimensi). Inilah inti dari filternya (misalnya matriks 3x3).
 - **public double Divisor:** Angka pembagi. Setelah semua angka dalam matriks dikalikan dengan pixel gambar dan dijumlahkan, hasilnya harus dibagi dengan angka ini. Ini penting untuk **Blur** agar gambar tidak menjadi terlalu terang (putih total).
 - **public int Offset:** Nilai penambah bias. Biasanya 0, tapi kadang digunakan untuk menaikkan kecerahan hasil akhir.
 - **Constructor (KernelData(...)):** Fungsi untuk mempermudah pembuatan objek ini dalam satu baris kode.
-

2. Database Generator (KernelDatabase)

Kelas statis ini berisi logika matematika untuk menentukan isi matriks berdasarkan nama filter yang dipilih user.

A. Inisialisasi

C#

```
public static KernelData GetKernel(string mode, int n)
{
    double[,] kernel = new double[n, n];
    double divisor = 1;
    int offset = 0;
    Random rnd = new Random();
    // ...
}
```

- **GetKernel:** Menerima mode (nama filter) dan n (ukuran matriks, misal 3 untuk 3x3).
- **double[,] kernel = new double[n, n]:** Menyiapkan matriks kosong seukuran $n \times n$.
- **double divisor = 1:** Default pembagi adalah 1 (tidak mengubah kecerahan).

B. Logika Filter (Per Baris Logika)

1. Mode: Random

Digunakan untuk efek noise atau gangguan sinyal.

C#

```
if (mode == "Random")
{
    for (int r = 0; r < n; r++)
        for (int c = 0; c < n; c++)
            kernel[r, c] = rnd.Next(-5, 6);
    divisor = 1;
}
```

- **Penjelasan:** Menggunakan *nested loop* (perulangan bersarang) untuk mengisi setiap sel matriks dengan angka acak antara -5 hingga 5 (karena Next pada C# bersifat *exclusive*

pada batas atas).

2. Mode: Blur (Box)

Efek buram standar (rata-rata).

C#

```
else if (mode.Contains("Blur (Box)"))
{
    for (int r = 0; r < n; r++)
        for (int c = 0; c < n; c++)
            kernel[r, c] = 1;
    divisor = n * n;
}
```

- **Matriks:** Semua sel diisi angka 1.
- **Divisor:** Diatur menjadi $n \times n$ (total jumlah elemen).
- **Logika:** Jika Anda menjumlahkan 9 pixel tetangga lalu membaginya dengan 9, Anda mendapatkan nilai **rata-rata** (Mean). Ini menghaluskan gambar.

3. Mode: Gaussian Blur

Efek buram yang lebih alami mengikuti kurva lonceng.

C#

```
else if (mode.Contains("Gaussian"))
{
    if (n == 3) {
        kernel = new double[,] { { 1, 2, 1 }, { 2, 4, 2 }, { 1, 2, 1 } };
        divisor = 16;
    }
    // else message box warning...
}
```

- **Hardcoded 3x3:** Kode ini secara spesifik mendefinisikan matriks Gaussian 3x3 yang populer.
- **Angka Matriks:** Pixel tengah memiliki bobot terbesar (4), tetangga terdekat (2), dan diagonal (1).
- **Divisor 16:** Didapat dari $1+2+1+2+4+2+1+2+1 = 16$. Jumlah total bobot harus dibagi

totalnya agar kecerahan gambar tetap sama.

4. Mode: Sharpen (Penajaman)

C#

```
else if (mode.Contains("Sharpen"))
{
    int mid = n / 2;
    kernel[mid, mid] = 5;
    // ... set tetangga jadi -1 ...
}
```

- **Konsep:** Pixel tengah diperkuat (dikalikan 5), sedangkan pixel tetangga dikurangi (dikalikan -1). Ini meningkatkan kontras lokal, membuat tepi terlihat lebih tegas.

5. Mode: Edge Detection (Simple & Laplacian)

Mendeteksi garis tepi objek.

- **Edge Simple:** Mirip Sharpen tapi lebih agresif (Tengah 4, tetangga -1). Menghasilkan gambar hitam dengan garis putih di tepian.
- **Laplacian:** Filter turunan kedua untuk deteksi tepi segala arah.

C#

```
kernel[n / 2, n / 2] = (n * n) - 1; // Tengah sangat positif
// Sisanya -1
```

Rumus ini memastikan jumlah total elemen adalah 0 (atau mendekati 0), sehingga area yang warnanya rata akan menjadi hitam, dan hanya perubahan warna drastis (tepi) yang menyala.

6. Mode: Sobel (Horizontal & Vertical)

Sobel sangat bagus untuk mendeteksi tepi berdasarkan arah bayangan.

C#

```
else if (mode.Contains("Sobel Horizontal"))
{
    if (n == 3)
        kernel = new double[,] { { -1, -2, -1 }, { 0, 0, 0 }, { 1, 2, 1 } };
```

```
// ... logika untuk ukuran 5 ...  
}
```

- **Sobel Horizontal:** Baris atas negatif, baris bawah positif. Ini mendeteksi perubahan warna arah vertikal (garis horizontal).
- **Sobel Vertical:** Kolom kiri negatif, kolom kanan positif. Mendeteksi garis vertikal.
- **Batasan:** Kode menggunakan `if (n==3) ... else if (n==5)` karena matriks Sobel memiliki angka spesifik yang sulit digenerate dengan *loop* sederhana, sehingga di-*hardcode*.

7. Mode: Emboss

Efek timbul seperti pahatan batu.

C#

```
else if (mode.Contains("Emboss"))  
{  
    // Contoh 3x3:  
    kernel = new double[,] { { -2, -1, 0 }, { -1, 1, 1 }, { 0, 1, 2 } };  
}
```

- **Logika:** Satu sisi diagonal negatif, sisi lainnya positif. Ini menciptakan efek bayangan dan *highlight* buatan, seolah-olah cahaya datang dari sudut tertentu.

C. Penanganan Error (ShowLimitWarning)

C#

```
private static void ShowLimitWarning(string featureName)  
{  
    MessageBox.Show( ... );  
}
```

- Metode ini dipanggil jika user mencoba menggunakan filter kompleks (seperti Sobel/Emboss) dengan ukuran kernel yang tidak didukung (selain 3x3 atau 5x5). Ini mencegah error *IndexOutOfRangeException* atau logika yang salah karena matriksnya di-*hardcode*.
-

Kesimpulan Singkat

Kode ini adalah **Pusat Logika Matematika** aplikasi Anda. Ia tidak memproses gambarnya secara langsung, melainkan **menyiapkan resepnya (Kernel)**.

1. User memilih "Blur".
2. Kode ini membuat matriks isi angka 1 dan divisor 9.
3. Objek `KernelData` dikirim ke fungsi lain (yang belum ada di sini) untuk dikalikan dengan pixel gambar asli.

Langkah Selanjutnya

Kode ini hanya mendefinisikan *Kernel*. Untuk melihat hasilnya pada gambar, Anda memerlukan algoritma **Konvolusi**.

Apakah Anda ingin saya buat kode untuk fungsi **Konvolusi**-nya (cara mengaplikasikan kernel ini ke objek `Bitmap`)?